

# Project 7

## Ethernet MAC (Lite Version)

### RTL Design and Verification

#### Problem Statement

Develop a lightweight Ethernet MAC (Media Access Controller) capable of handling Ethernet frame transmission and reception with basic error detection. The design implements a full-duplex MAC using the RMII (Reduced Media Independent Interface) to interface with an external PHY. Both TX and RX datapaths, a CRC-32 engine, and a preamble/SFD generator/stripper are implemented at RTL level in Verilog HDL, verified through simulation, and validated on FPGA.

#### Project Milestones

##### Milestone 1: Problem Analysis & System Design

Define MAC functionality and Ethernet frame structure. Specify RMII interface signals, clock domains, frame format (preamble, SFD, DA, SA, EtherType/Length, payload, FCS), minimum/maximum frame sizes, and full-duplex operation assumptions.

##### Milestone 2: RTL Design & Implementation

Implement transmit and receive datapaths in Verilog HDL. Design the CRC-32 engine, preamble generator/stripper, TX FIFO interface, RX FIFO interface, and top-level integration module.

##### Milestone 3: Verification & FPGA Validation

Validate frame handling and error detection through self-checking testbench simulation. Test normal frames, runt/giant frame rejection, CRC error detection, and back-to-back frame reception. Document results.

#### System Architecture

##### Ethernet Frame Format

Each Ethernet II frame consists of: 7-byte Preamble (0x55), 1-byte SFD (0xD5), 6-byte Destination Address, 6-byte Source Address, 2-byte EtherType/Length, 46–1500 byte Payload, and 4-byte FCS (CRC-32). Minimum frame size: 64 bytes (excluding preamble/SFD). Maximum frame size: 1518 bytes.

##### System Parameters

| Parameter      | Value       | Description                           |
|----------------|-------------|---------------------------------------|
| System Clock   | 50 MHz      | FPGA system clock input               |
| RMII Ref Clock | 50 MHz      | Reference clock to PHY (REF_CLK)      |
| Interface      | RMII        | 2-bit data path to/from PHY at 50 MHz |
| Duplex         | Full-duplex | No CSMA/CD; simultaneous TX and RX    |

|           |              |                                  |
|-----------|--------------|----------------------------------|
| Min Frame | 64 bytes     | Excluding preamble and SFD       |
| Max Frame | 1518 bytes   | Excluding preamble and SFD       |
| CRC       | CRC-32       | IEEE 802.3 polynomial 0x04C11DB7 |
| Preamble  | 7 bytes 0x55 | Generated on TX; stripped on RX  |
| SFD       | 0xD5         | Start Frame Delimiter            |
| IFG       | 96 bit times | Inter-frame gap enforced on TX   |

## RTL Design — Verilog Source Code

---

### 1. CRC-32 Engine

Computes the IEEE 802.3 CRC-32 over incoming/outgoing bytes. On TX the residue is appended as FCS; on RX the residue over data+FCS equals the magic constant 0xC704DD7B when no error is present.

```
// =====
// Module : crc32.v
// Purpose : IEEE 802.3 CRC-32 byte-serial engine
// =====
module crc32 (
    input wire      clk,
    input wire      rst_n,
    input wire      init,      // Synchronous reset to 0xFFFFFFFF
    input wire      en,        // Process one byte per cycle
    input wire [7:0] data_in,  // Byte input (LSB first per 802.3)
    output wire [31:0] crc_out, // Current CRC value
    output wire [31:0] crc_fcs  // Byte-reversed, bit-inverted FCS
);
    reg [31:0] crc_reg;

    function [31:0] crc32_byte;
        input [31:0] crc;
        input [7:0] data;
        integer i;
        reg [31:0] c;
        reg      inv;
        begin
            c = crc;
            for (i = 0; i < 8; i = i + 1) begin
                inv = c[31] ^ data[i];
                c = {c[30:0], 1'b0} ^ (inv ? 32'h04C11DB7 : 32'h0);
            end
            crc32_byte = c;
        end
    endfunction

    always @(posedge clk) begin
        if (!rst_n || init)
            crc_reg <= 32'hFFFF_FFFF;
        else if (en)
            crc_reg <= crc32_byte(crc_reg, data_in);
    end

    assign crc_out = crc_reg;
    // FCS: bit-reverse each byte then invert all bits
    assign crc_fcs = ~{
        {crc_reg[24],crc_reg[25],crc_reg[26],crc_reg[27],
         crc_reg[28],crc_reg[29],crc_reg[30],crc_reg[31]},
        {crc_reg[16],crc_reg[17],crc_reg[18],crc_reg[19],
         crc_reg[20],crc_reg[21],crc_reg[22],crc_reg[23]},
        {crc_reg[8], crc_reg[9], crc_reg[10],crc_reg[11],
         crc_reg[12],crc_reg[13],crc_reg[14],crc_reg[15]},
        {crc_reg[0], crc_reg[1], crc_reg[2], crc_reg[3],
         crc_reg[4], crc_reg[5], crc_reg[6], crc_reg[7]}}
    };
endmodule
```

## 2. Ethernet MAC Transmitter (TX)

Implements the TX datapath FSM: IDLE → PREAMBLE → SFD → DATA → FCS → IFG. Reads bytes from the TX FIFO, prepends preamble and SFD, appends the CRC-32 FCS, and serialises to the RMIi 2-bit interface at 50 MHz (100 Mbps).

```
// =====
// Module : mac_tx.v
// Purpose : Ethernet MAC Transmitter – RMIi interface
// =====
module mac_tx (
    input wire      clk,          // 50 MHz system / RMIi clock
    input wire      rst_n,        // Active-low sync reset
    // RMIi PHY interface
    output reg [1:0] rmi_txd,     // 2-bit TX data to PHY
    output reg       rmi_tx_en,   // TX enable to PHY
    // FIFO interface (host side)
    input wire [7:0] tx_fifo_data, // Byte from TX FIFO
    input wire       tx_fifo_empty,
    output reg       tx_fifo_rd,   // Read strobe
    // Control / status
    input wire       tx_start,     // Begin transmission
    output reg       tx_busy,
    output reg       tx_done
);
// State encoding
localparam IDLE     = 3'd0;
localparam PREAMBLE = 3'd1;
localparam SFD      = 3'd2;
localparam DATA    = 3'd3;
localparam FCS      = 3'd4;
localparam IFG      = 3'd5;

localparam IFG_NIBBLES = 24; // 96 bits / 4 bits per RMIi pair = 24 cycles

reg [2:0] state;
reg [3:0] preamble_cnt; // 0..6 (7 preamble bytes)
reg [1:0] dibit_cnt;    // 0..3 (4 dibits per byte)
reg [7:0] byte_reg;     // Current byte being serialised
reg [4:0] ifg_cnt;
reg [1:0] fcs_byte_idx; // 0..3 (4 FCS bytes)

// CRC engine wires
reg      crc_init, crc_en;
reg [7:0] crc_data;
wire [31:0] crc_out, crc_fcs;

crc32 u_crc (
    .clk(clk), .rst_n(rst_n),
    .init(crc_init), .en(crc_en),
    .data_in(crc_data),
    .crc_out(crc_out), .crc_fcs(crc_fcs)
);

always @(posedge clk) begin
    if (!rst_n) begin
        state      <= IDLE;
        rmi_txd     <= 2'b00;
        rmi_tx_en   <= 1'b0;
        tx_fifo_rd  <= 1'b0;
        tx_busy     <= 1'b0;
        tx_done     <= 1'b0;
        crc_init    <= 1'b0;
        crc_en      <= 1'b0;
        preamble_cnt <= 4'd0;
    end
end
```

```

dibit_cnt    <= 2'd0;
ifg_cnt      <= 5'd0;
fcs_byte_idx <= 2'd0;
end else begin
tx_done      <= 1'b0;
tx_fifo_rd   <= 1'b0;
crc_en       <= 1'b0;
crc_init     <= 1'b0;

case (state)
IDLE: begin
rmii_tx_en <= 1'b0;
rmii_txd   <= 2'b00;
tx_busy    <= 1'b0;
if (tx_start && !tx_fifo_empty) begin
crc_init    <= 1'b1;
preamble_cnt <= 4'd0;
dibit_cnt   <= 2'd0;
tx_busy     <= 1'b1;
state       <= PREAMBLE;
end
end

PREAMBLE: begin
rmii_tx_en <= 1'b1;
rmii_txd   <= 2'b01; // 0x55 = 01_01_01_01 repeated
dibit_cnt  <= dibit_cnt + 1;
if (dibit_cnt == 2'd3) begin
if (preamble_cnt == 4'd6) begin
dibit_cnt <= 2'd0;
state     <= SFD;
end else
preamble_cnt <= preamble_cnt + 1;
end
end

SFD: begin
// SFD = 0xD5 = 11_01_01_01 (LSB first)
rmii_tx_en <= 1'b1;
case (dibit_cnt)
2'd0: rmii_txd <= 2'b01;
2'd1: rmii_txd <= 2'b01;
2'd2: rmii_txd <= 2'b01;
2'd3: rmii_txd <= 2'b11;
endcase
dibit_cnt <= dibit_cnt + 1;
if (dibit_cnt == 2'd3) begin
dibit_cnt <= 2'd0;
tx_fifo_rd <= 1'b1;
state     <= DATA;
end
end

DATA: begin
rmii_tx_en <= 1'b1;
rmii_txd   <= byte_reg[1:0]; // LSB first
byte_reg   <= {2'b00, byte_reg[7:2]};
dibit_cnt  <= dibit_cnt + 1;
if (dibit_cnt == 2'd3) begin
crc_en     <= 1'b1;
crc_data   <= tx_fifo_data; // CRC over current byte
if (tx_fifo_empty) begin
dibit_cnt <= 2'd0;
fcs_byte_idx <= 2'd0;
state     <= FCS;
end
end
end

```

```

        end else begin
            tx_fifo_rd <= 1'b1;
            byte_reg <= tx_fifo_data;
        end
    end
end

FCS: begin
    rmii_tx_en <= 1'b1;
    case (fcs_byte_idx)
        2'd0: byte_reg <= crc_fcs[7:0];
        2'd1: byte_reg <= crc_fcs[15:8];
        2'd2: byte_reg <= crc_fcs[23:16];
        2'd3: byte_reg <= crc_fcs[31:24];
    endcase
    rmii_txd <= byte_reg[1:0];
    byte_reg <= {2'b00, byte_reg[7:2]};
    dibit_cnt <= dibit_cnt + 1;
    if (dibit_cnt == 2'd3) begin
        dibit_cnt <= 2'd0;
        if (fcs_byte_idx == 2'd3) begin
            ifg_cnt <= 5'd0;
            state <= IFG;
        end else
            fcs_byte_idx <= fcs_byte_idx + 1;
        end
    end
end

IFG: begin
    rmii_tx_en <= 1'b0;
    rmii_txd <= 2'b00;
    ifg_cnt <= ifg_cnt + 1;
    if (ifg_cnt == IFG_NIBBLES - 1) begin
        tx_done <= 1'b1;
        state <= IDLE;
    end
end

default: state <= IDLE;
endcase
end
end

// Latch FIFO data on read
always @(posedge clk) begin
    if (tx_fifo_rd)
        byte_reg <= tx_fifo_data;
end
endmodule

```

### 3. Ethernet MAC Receiver (RX)

Detects incoming RMII frames by watching for preamble/SFD patterns, deserialises dibits into bytes, accumulates a running CRC-32, and on stop outputs the frame to the RX FIFO with a valid/error status pulse. A 2-FF synchroniser protects against metastability on RXD inputs.

```
// =====
// Module : mac_rx.v
// Purpose : Ethernet MAC Receiver – RMII interface
// =====
module mac_rx (
    input wire      clk,          // 50 MHz RMII clock
    input wire      rst_n,
    // RMII PHY interface
    input wire [1:0] rmii_rxd,    // 2-bit RX data from PHY
    input wire      rmii_crs_dv,  // Carrier sense / data valid
    // FIFO interface (host side)
    output reg [7:0] rx_fifo_data, // Byte to RX FIFO
    output reg      rx_fifo_wr,    // Write strobe
    // Status
    output reg      rx_done,       // Frame complete (good or bad)
    output reg      crc_error,     // FCS mismatch
    output reg      frame_error    // Runt / giant frame
);

    localparam IDLE   = 3'd0;
    localparam PREAM  = 3'd1;
    localparam SFD     = 3'd2;
    localparam DATA   = 3'd3;
    localparam STATUS  = 3'd4;

    localparam MIN_BYTES = 12'd60; // Min payload (excl. FCS)
    localparam MAX_BYTES = 12'd1514; // Max payload (excl. FCS)

    reg [2:0] state;
    reg [1:0] rxd_s1, rxd_s2; // 2-FF synchroniser
    reg      dv_s1, dv_s2;
    reg [1:0] dibit_cnt;
    reg [7:0] byte_shift;
    reg [11:0] byte_cnt; // Bytes received in DATA state
    reg [7:0] prev_byte; // Pipeline for CRC boundary

    // CRC engine
    reg      crc_init, crc_en;
    wire [31:0] crc_out, crc_fcs;

    crc32 u_crc (
        .clk(clk), .rst_n(rst_n),
        .init(crc_init), .en(crc_en),
        .data_in(byte_shift),
        .crc_out(crc_out), .crc_fcs(crc_fcs)
    );

    // 2-FF synchroniser for RMII inputs
    always @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            rxd_s1 <= 2'b0; rxd_s2 <= 2'b0;
            dv_s1 <= 1'b0; dv_s2 <= 1'b0;
        end else begin
            rxd_s1 <= rmii_rxd; rxd_s2 <= rxd_s1;
            dv_s1 <= rmii_crs_dv; dv_s2 <= dv_s1;
        end
    end

    always @(posedge clk) begin
        if (!rst_n) begin
```

```

state      <= IDLE;
dibit_cnt  <= 2'd0;
byte_cnt   <= 12'd0;
rx_fifo_wr <= 1'b0;
rx_done    <= 1'b0;
crc_error  <= 1'b0;
frame_error <= 1'b0;
crc_init   <= 1'b0;
crc_en     <= 1'b0;
end else begin
rx_fifo_wr <= 1'b0;
rx_done    <= 1'b0;
crc_error  <= 1'b0;
frame_error <= 1'b0;
crc_init   <= 1'b0;
crc_en     <= 1'b0;

case (state)
IDLE: begin
    if (dv_s2 && rxd_s2 == 2'b01) begin
        // First preamble dibit detected
        dibit_cnt <= 2'd1;
        state     <= PREAM;
    end
end

PREAM: begin
    if (!dv_s2) begin state <= IDLE; end
    else if (rxd_s2 == 2'b01) begin
        dibit_cnt <= dibit_cnt + 1; // keep counting 0x55 dibits
    end else if (rxd_s2 == 2'b11) begin
        // SFD first dibit (0xD5 LSB nibble = 2'b11)
        state <= SFD;
    end else state <= IDLE; // invalid
end

SFD: begin
    // Remaining 3 dibits of SFD already consumed, go to DATA
    if (!dv_s2) begin state <= IDLE; end
    else begin
        crc_init <= 1'b1;
        byte_cnt <= 12'd0;
        dibit_cnt <= 2'd0;
        state <= DATA;
    end
end

DATA: begin
    if (!dv_s2) begin
        // End of frame – check byte count
        state <= STATUS;
    end else begin
        // Shift dibits into byte (LSB first)
        byte_shift <= {rxd_s2, byte_shift[7:2]};
        dibit_cnt <= dibit_cnt + 1;
        if (dibit_cnt == 2'd3) begin
            crc_en <= 1'b1;
            rx_fifo_data <= {rxd_s2, byte_shift[7:2]};
            rx_fifo_wr <= 1'b1;
            byte_cnt <= byte_cnt + 1;
        end
    end
end

STATUS: begin

```

```

        rx_done <= 1'b1;
        // CRC-32 residue over data+FCS must equal 0xC704DD7B
        crc_error <= (crc_out != 32'hC704DD7B);
        // Frame size check (byte_cnt includes 4 FCS bytes)
        frame_error <= (byte_cnt < (MIN_BYTES + 4)) ||
                        (byte_cnt > (MAX_BYTES + 4));

        state <= IDLE;
    end

    default: state <= IDLE;
endcase

end

end
endmodule

```

#### 4. Ethernet MAC Top-Level Module

Instantiates and wires together the CRC-32 engine (shared), TX module, and RX module into a complete self-contained Ethernet MAC Lite controller.

```
// =====
// Module : mac_top.v
// Purpose : Top-level Ethernet MAC Lite Controller
// =====
module mac_top (
    // Global
    input wire      clk,          // 50 MHz
    input wire      rst_n,        // Active-low sync reset
    // RMI PHY interface - TX
    output wire [1:0] rmii_txd,
    output wire      rmii_tx_en,
    // RMI PHY interface - RX
    input wire [1:0] rmii_rxd,
    input wire      rmii_crs_dv,
    // TX FIFO interface (host)
    input wire [7:0] tx_fifo_data,
    input wire      tx_fifo_empty,
    output wire      tx_fifo_rd,
    input wire      tx_start,
    output wire      tx_busy,
    output wire      tx_done,
    // RX FIFO interface (host)
    output wire [7:0] rx_fifo_data,
    output wire      rx_fifo_wr,
    output wire      rx_done,
    output wire      crc_error,
    output wire      frame_error
);

// Transmitter
mac_tx u_tx (
    .clk          (clk),
    .rst_n        (rst_n),
    .rmii_txd     (rmii_txd),
    .rmii_tx_en   (rmii_tx_en),
    .tx_fifo_data (tx_fifo_data),
    .tx_fifo_empty (tx_fifo_empty),
    .tx_fifo_rd   (tx_fifo_rd),
    .tx_start     (tx_start),
    .tx_busy      (tx_busy),
    .tx_done      (tx_done)
);

// Receiver
mac_rx u_rx (
    .clk          (clk),
    .rst_n        (rst_n),
    .rmii_rxd     (rmii_rxd),
    .rmii_crs_dv  (rmii_crs_dv),
    .rx_fifo_data (rx_fifo_data),
    .rx_fifo_wr   (rx_fifo_wr),
    .rx_done      (rx_done),
    .crc_error    (crc_error),
    .frame_error  (frame_error)
);
endmodule
```

The testbench models a PHY loopback: frames driven into the TX FIFO are looped back to the RX RMII input. It verifies end-to-end data integrity, CRC pass/fail, and frame error detection across multiple test vectors.

[illegible]

[illegible]

[illegible]

## 6. Simulation & FPGA Flow Commands

---

### Icarus Verilog (iverilog) Simulation

```
# Compile all source files + testbench
iverilog -o mac_sim \
    crc32.v mac_tx.v mac_rx.v mac_top.v tb_mac_top.v

# Run simulation
vvp mac_sim

# View waveform (GTKWave)
gtkwave mac_sim.vcd &
```

### Xilinx Vivado FPGA Flow

```
# Synthesize (Tcl console)
read_verilog crc32.v mac_tx.v mac_rx.v mac_top.v
synth_design -top mac_top -part xc7a35tcpg236-1

# Run implementation
opt_design
place_design
route_design

# Generate bitstream
write_bitstream -force mac_top.bit

# Program FPGA
open_hw_manager
connect_hw_server
program_hw_devices [get_hw_devices]
```

### Intel Quartus FPGA Flow

```
# Create project and compile
package require ::quartus::project
project_new mac_top -overwrite
set_global_assignment -name TOP_LEVEL_ENTITY mac_top
set_global_assignment -name VERILOG_FILE mac_top.v
set_global_assignment -name VERILOG_FILE mac_tx.v
set_global_assignment -name VERILOG_FILE mac_rx.v
set_global_assignment -name VERILOG_FILE crc32.v
execute_flow -compile
project_close
```

## 7. Verification Checklist & Expected Results

| Test Case                   | Expected Behaviour                      | Status |
|-----------------------------|-----------------------------------------|--------|
| Min frame (46-byte payload) | rx_done=1, no crc_error, no frame_error | Verify |
| 100-byte payload            | rx_done=1, data matches TX FIFO         | Verify |
| Max frame (1500-byte)       | rx_done=1, no errors                    | Verify |
| All-zero payload            | CRC-32 computed correctly               | Verify |
| All-0xFF payload            | CRC-32 computed correctly               | Verify |
| CRC error injection         | crc_error = 1 on bad FCS                | Verify |
| Runt frame (<64 bytes)      | frame_error = 1                         | Verify |
| Giant frame (>1518 bytes)   | frame_error = 1                         | Verify |
| Back-to-back frames         | No frame data loss; IFG respected       | Verify |
| Reset during TX             | TX returns to IDLE, rmii_tx_en=0        | Verify |
| Preamble/SFD detection      | DATA state entered correctly            | Verify |
| False start (no SFD)        | RX returns to IDLE                      | Verify |

## 8. Project File Structure

```
ethernet_mac_lite/  
|-- rtl/  
|   |-- crc32.v          # IEEE 802.3 CRC-32 byte-serial engine  
|   |-- mac_tx.v         # Ethernet MAC Transmitter FSM (RMII)  
|   |-- mac_rx.v         # Ethernet MAC Receiver FSM (RMII)  
|   |-- mac_top.v        # Top-level integration module  
|-- tb/  
|   |-- tb_mac_top.v     # Self-checking testbench (PHY loopback)  
|-- constraints/  
|   |-- mac_top.xdc      # Xilinx pin constraints (Basys3 / Arty)  
|-- sim/  
|   |-- mac_sim.vcd      # Simulation waveform output  
|-- docs/  
|   |-- Ethernet_MAC_Lite_Project.pdf # This document
```